High-Order Winslow smoothing/untangling

Andrew Shi

April 27, 2026

Contents

0 The 2016 formulation (brief recap) 2
0.1 Numerical Results/Tests . . . 2

1 W modification to achieve reference element shape 3
1.1 Target metric . . . 3
1.2 Modified split form . . . 5
1.3 Continuity of W and the $\tilde{\alpha}$ projection . . . 5
1.4 Numerical Results/Tests . . . 6
1.5 Summary . . . 8

2 Reference-mesh preprocessing via Tutte's embedding 9
2.1 Extracting the topology . . . 10
2.2 Tutte's barycentric embedding . . . 10
2.3 Shape optimization of the straight-sided mesh . . . 11
2.4 Promotion to high-order reference . . . 12
2.5 Numerical examples . . . 12
2.5.1 Example 1: regular grid, `w2.m` topology . . . 12
2.5.2 Example 2: unstructured Delaunay on the unit square . . . 12
2.5.3 Example 3: convex stadium . . . 14
2.5.4 Example 4: smooth non-convex starfish . . . 14
2.5.5 Example 5: non-convex L-shape, two target shapes . . . 18
2.5.6 Example 6: comb (cooperative reference shape) . . . 22

Codes live here.

0 The 2016 formulation (brief recap)

The Winslow equations in physical coordinates are

$$g^{ij}\,\partial_i\partial_j x_k = 0, \qquad k = 1, \ldots, n, \tag{1}$$

where g^{ij} are the contravariant metric components of the (nonlinear) mapping from the computational domain C to the physical domain D. The nonconservative form does not discretize cleanly in a CG framework, so the 2016 paper introduces auxiliary variables $\alpha_j = -\partial_i(g^{ij})$ and recasts the system in split form:

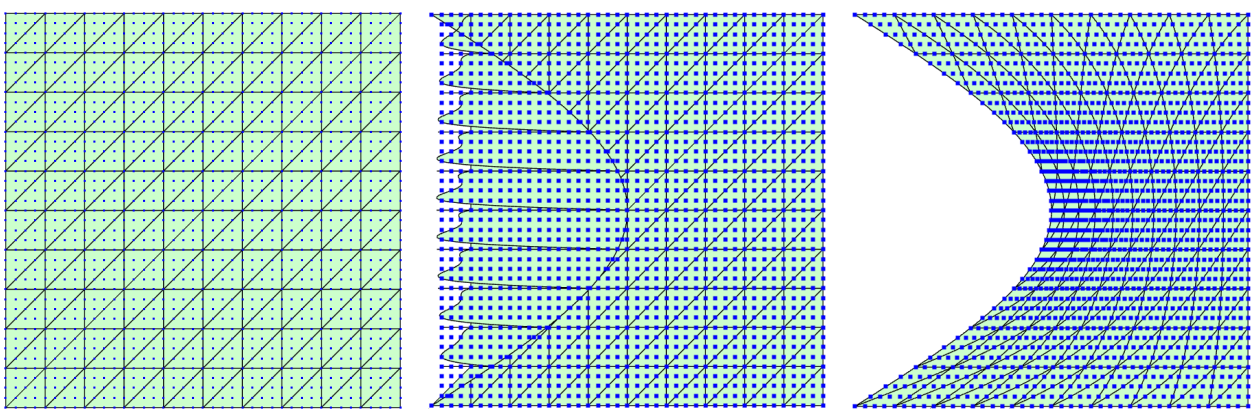
$$\partial_i(g^{ij}) + \alpha_j = 0, \tag{2}$$

$$\partial_i(g^{ij}\,\partial_j x_k) + \alpha_j\,\partial_j x_k = 0, \tag{3}$$

with Dirichlet data $\boldsymbol{x} = \boldsymbol{x}_{\text{bnd}}$ on ∂C. The point of this split form is to 1) mimic the what finite difference Winslow people are doing and 2) enable Picard iteration, which is the "magic" here.

0.1 Numerical Results/Tests

The code `w2.m` does the following in Figure 1.



(a) Reference mesh. (b) Initial configuration. (c) After Winslow

Figure 1: (a) Structured 11×11 reference mesh at $p = 4$. (b) Initial configuration: interior nodes unchanged, boundary on side 4 displaced by $0.5\sin(\pi y)$, producing heavy tangling. (c) After Winslow iterations, interior nodes have redistributed to absorb the boundary deformation; all elements valid.

To assess mesh quality we use the scaled Jacobian of Fortunato–Persson [1],

$$I \;=\; \frac{\min_{\xi\in\kappa} J(\xi)}{\max_{\xi\in\kappa} J(\xi)}, \qquad I \le 1, \tag{4}$$

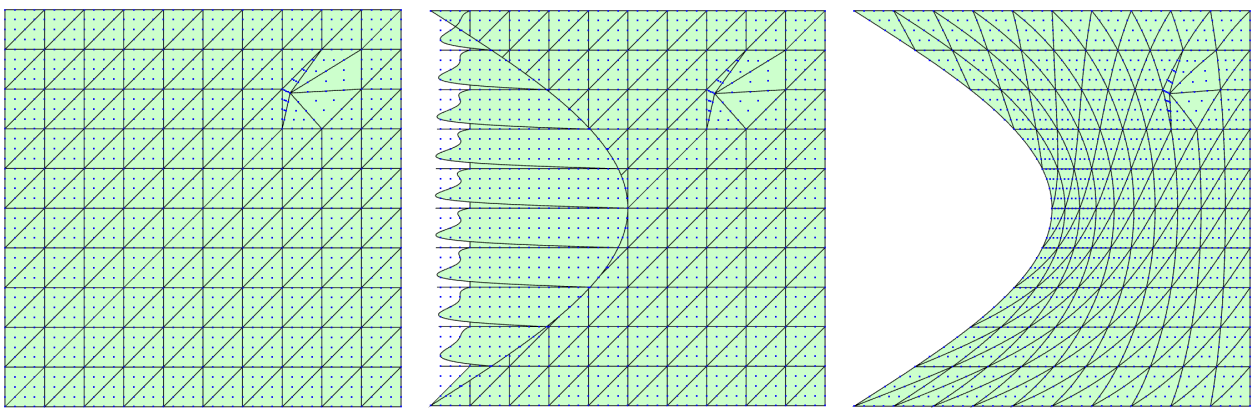
approximated at the Gauss quadrature points of each element κ. For a straight-sided simplex, J is constant across κ and $I = 1$ exactly; $I \le 0$ indicates a degenerate or inverted element.

To set up the main test case we will use throughout Section 1, we introduce `w2_skewed.m`, a slight variation of `w2.m` in which a single interior linear vertex of the otherwise uniform 11×11

reference is moved toward a neighbor, creating a small patch of sliver elements in the upper-right of the reference. Everything else is identical to `w2.m`. The scaled Jacobian I of (4) is blind to this regime: any simplex with straight edges has $I = 1$ regardless of aspect ratio, so it cannot "see" the sliver patch in the reference. To capture straight-sided shape distortion we also report the Liu–Joe inverse mean-ratio shape distortion metric [2, 3],

$$\eta \;=\; \frac{\|\boldsymbol{J}\|_F^2}{d\,|\det \boldsymbol{J}|^{2/d}}, \qquad \eta \in [1, \infty), \tag{5}$$

with $\eta = 1$ for a conformal (ideal) element and $\eta \to \infty$ as the element degenerates. The results are summarized in Figure 2 (meshes), Figure 3 (scaled Jacobian distribution), and Figure 4 (shape distortion distribution). Unmodified Winslow produces a valid physical mesh, but the sliver cluster visible in the reference persists as the worst element in the smoothed output. This is exactly the behavior the W-modification of Section 1 is designed to address, and the single-sliver `w2_skewed` test is the setting in which we evaluate it.



(a) Reference mesh. (b) Initial configuration. (c) After Winslow

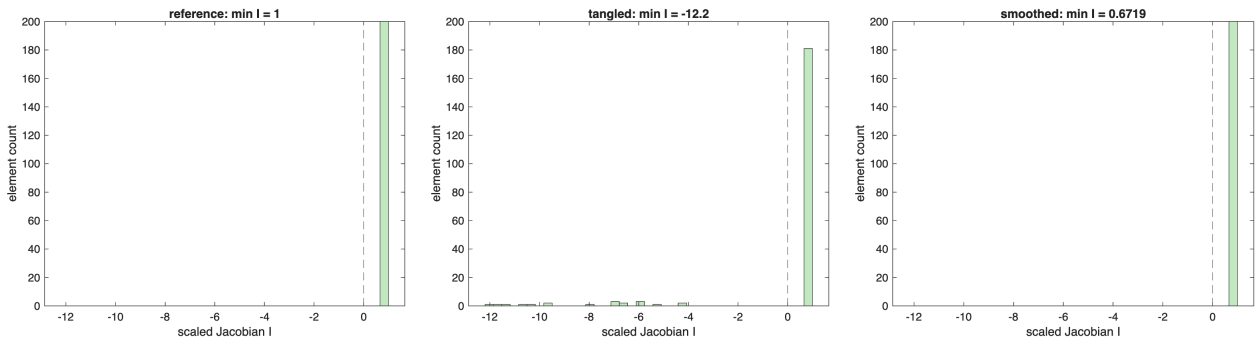
Figure 2: Slight variation of `w2.m`: one interior vertex of the 11×11 reference at $p = 4$ is displaced toward a neighbor, creating a sliver cluster. (a) Skewed reference with $\max \eta = 4.63$. (b) Boundary on side 4 displaced by $0.5 \sin(\pi y)$, producing heavy tangling near the perturbed edge on top of the reference-induced sliver. (c) After unmodified Winslow: elements are valid, but the reference skew persists, with $\max \eta = 7.17$ on the sliver in the upper-right.

1 W modification to achieve reference element shape

The 2016 formulation produces a physical mesh whose element shapes are inherited from the reference mesh: the contravariant metric g^{ij} in the split-form system of Section 0 is built from the reference element geometry, so a skewed reference produces skewed physical elements. The W-modification below decouples the target shape from the reference element shape. **This is exactly the same formulation Persson gave me from his Gemini conversation (persson_gemini_winslow.pdf).**

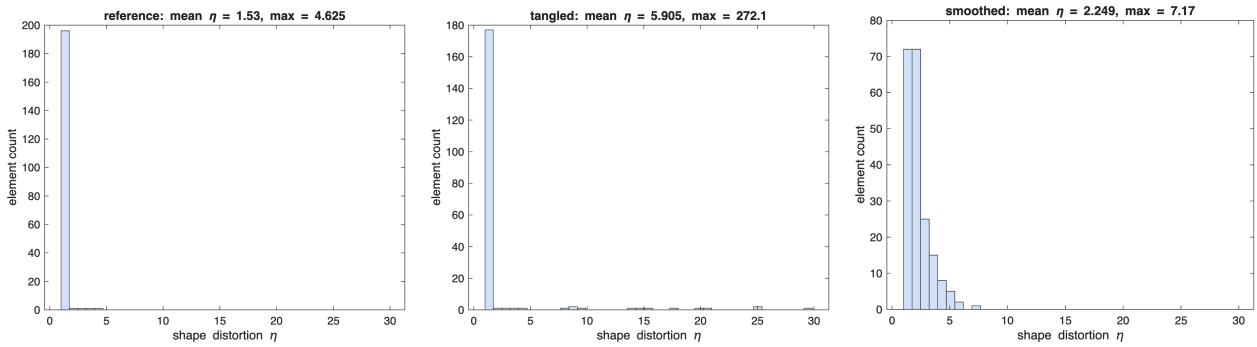
1.1 Target metric

For each reference element $K \in \mathcal{T}_h$, let $W_K \in \mathbb{R}^{n \times n}$ denote the Jacobian of an affine map from an ideal element (e.g. a unit equilateral simplex or unit square) to the reference element geometry.



(a) Reference (b) Tangled (c) Smoothed

Figure 3: Per-element scaled Jacobian $I = \min_\xi J(\xi)/\max_\xi J(\xi)$ for the `w2_skewed` test. (a) Reference: all elements straight-sided simplex, so $I = 1$ identically—the metric is blind to the sliver cluster present in the mesh. (b) Tangled: long negative tail, $\min I = -12.2$. (c) Smoothed: all $I > 0$, $\min I \approx 0.67$ on elements bordering the indentation.



(a) Reference (b) Tangled (c) Smoothed

Figure 4: Per-element shape distortion metric $\eta = \|J\|_F^2/(d\,|\det J|^{2/d})$ [2, 3] for the `w2_skewed` test. $\eta \in [1, \infty)$ with $\eta = 1$ at conformal (ideal) elements; unlike I, η detects straight-sided slivers. (a) Reference: $\max \eta = 4.63$, mean 1.53—the right tail is the sliver cluster around the moved vertex. (b) Tangled: $\max \eta = 272$. (c) Smoothed: $\max \eta = 7.17$, mean 2.25—the sliver cluster persists as the worst element in the smoothed output, motivating the W-modification in Section 1.

The total map from the ideal element to the physical element then has Jacobian $J_{\text{tot}} = JW_K^{-1}$, where $J = \partial x/\partial \xi$ is the reference-to-physical Jacobian.

The covariant and contravariant metrics of the total map are

$$\tilde{g}_{ij} = (J_{\text{tot}}^T J_{\text{tot}})_{ij} = (W_K^{-T} g W_K^{-1})_{ij}, \qquad \tilde{g}^{ij} = (W_K g^{-1} W_K^T)_{ij} = W_{K,ik}\, g^{kl}\, W_{K,jl}. \tag{6}$$

With $W_K = I$ for all K, $\tilde{g}^{ij} = g^{ij}$ and the 2016 formulation is recovered.

To summarize the three Jacobians introduced above and what they map between:

- $J = \partial x/\partial \xi$: reference $\to$ physical. The Winslow Jacobian of Section 0.
- W_K: reference $\to$ ideal. Per-element matrix recording how skewed the reference element is relative to the ideal target shape; $W_K = I$ means the reference element is already ideal.
- $J_{\text{tot}} = J\, W_K^{-1}$: ideal $\to$ physical. Composite Jacobian of the chain ideal $\to$ reference $\to$ physical, whose departure from the identity measures how far the physical element deviates from the ideal target shape.

1.2 Modified split form

We replace g^{ij} by $\tilde{g}^{ij}$ in the split-form system of Section 0, introducing a modified auxiliary variable $\tilde{\alpha}_j = -\partial_i \tilde{g}^{ij}$:

$$\partial_i(\tilde{g}^{ij}) + \tilde{\alpha}_j = 0, \tag{7}$$

$$\partial_i(\tilde{g}^{ij}\, \partial_j x_k) + \tilde{\alpha}_j\, \partial_j x_k = 0, \tag{8}$$

with Dirichlet data $x = x_{\text{bnd}}$ on ∂C. The equivalence with the nonconservative form $\tilde{g}^{ij}\partial_i\partial_j x_k = 0$ follows from (7) by the same product-rule argument as in the unmodified case.

Heuristic nature of the extension. The classical Winslow system $g^{ij}\partial_i\partial_j x_k = 0$ is the reformulation, via change of variables, of $\Delta_x \xi_k = 0$—the Euler–Lagrange equation of the Dirichlet energy $\int_D |\nabla_x \xi|^2\, dx$ for the inverse map. The substitution $g \mapsto \tilde{g}$ in (7)–(8) is not, to the authors' knowledge, the Euler–Lagrange equation of any natural energy functional in x. In particular, for piecewise-constant W_K the map $\xi \mapsto W_K \xi$ is a per-element linear reparameterization of the reference coordinates, and harmonic maps are invariant under such reparameterizations, so the natural variational interpretation collapses to the unmodified Winslow system. We therefore regard (7)–(8) as a heuristic extension of the 2016 numerical scheme, parameterized so as to retain its Picard structure, rather than as the discretization of a derived PDE. Consequences: no Radó–Kneser–Choquet analog applies, and convergence of the resulting Picard iteration is assessed empirically.

1.3 Continuity of W and the $\tilde{\alpha}$ projection

The original Fortunato–Persson formulation already solves one projection problem: the auxiliary variable α_j is defined, through the weak form of $\partial_i g^{ij} + \alpha_j = 0$, as the L^2 projection of $-\partial_i g^{ij}$ onto the continuous CG space V_h^p. Operationally this is the linear solve $\alpha \leftarrow M\backslash b$, where M is the V_h^p mass matrix and b is assembled by quadrature from g^{ij} at Gauss points. The solve is what makes the scheme work on unstructured meshes. Because $x_h \in V_h^p$ is C^0 but not C^1, the gradient $\partial x/\partial \xi$ jumps across every internal edge, so g^{ij} is discontinuous there, and a strong derivative of g^{ij} would produce Dirac-style spikes supported on the interfaces. By omitting internal face fluxes in

the weak form, the projection smears those spikes into a single continuous $\alpha \in V_h^p$, bypassing the DG machinery of numerical fluxes entirely. This is the computational magic of the 2016 paper.

In the modified system (7)–(8) this projection is reused without modification. $\tilde{\alpha}_j$ is the V_h^p projection of $-\partial_i \tilde{g}^{ij}$, computed with the *same* mass matrix M and the same backslash solve; only the right-hand side is altered to use $\tilde{g}^{ij} = W_K g^{ij} W_K^T$ instead of g^{ij}. No new projection infrastructure is introduced at this stage, and one can formally proceed without any further thought about the role of W_K.

The genuinely new object is W_K. It is the Jacobian of the affine map from the ideal equilateral triangle to the (straight-sided) reference triangle K—a map that sends the three corners of the ideal element to the three corners of K. Because the map is affine, its Jacobian W_K is a *single* 2×2 matrix, constant throughout the interior of K, at every polynomial order p; the polynomial order p controls how the *solution* x_h is represented, not the geometry of K, and the straight-sided reference keeps W_K element-constant regardless of p. The subscript K records the essential point that W_K is one matrix per element and jumps discontinuously across interfaces, since adjacent reference triangles are mapped from the ideal by different affine transformations.

At a glance it is tempting to conclude that these new jumps are absorbed for free by the $\tilde{\alpha}$-projection, just as the old jumps of g^{ij} were. The projection is indeed well-defined, but a subtler point intervenes. The jumps of g^{ij} at an interface are controlled by the jumps of $\partial x_h / \partial \xi$ there, and these shrink to zero as the Picard iteration converges, since the fixed point x_h is as smooth as the discrete space allows. FP's robustness relies on this self-limiting behavior: the coefficient roughness that the iteration has to cope with decays alongside the iteration itself. The jumps of W_K are structurally different. They are set once and for all by the initial reference mesh, are external to the iterate, and *do not* decay with Picard. If the reference mesh contains neighbors of very different shape, the W_K jumps can be substantial, and we have observed empirically that the resulting persistent coefficient roughness destabilizes the iteration.

The remedy suggested in the Persson/Gemini exchange is to elevate W to V_h^p *before* forming $\tilde{g}^{ij}$. At each CG node x_n, assemble a nodal target $W_h(x_n)$ as a weighted average of the W_K values from the surrounding elements,

$$W_h(\mathbf{x}_n) = \frac{\sum_{K \ni \mathbf{x}_n} |K| \, W_K}{\sum_{K \ni \mathbf{x}_n} |K|}, \tag{9}$$

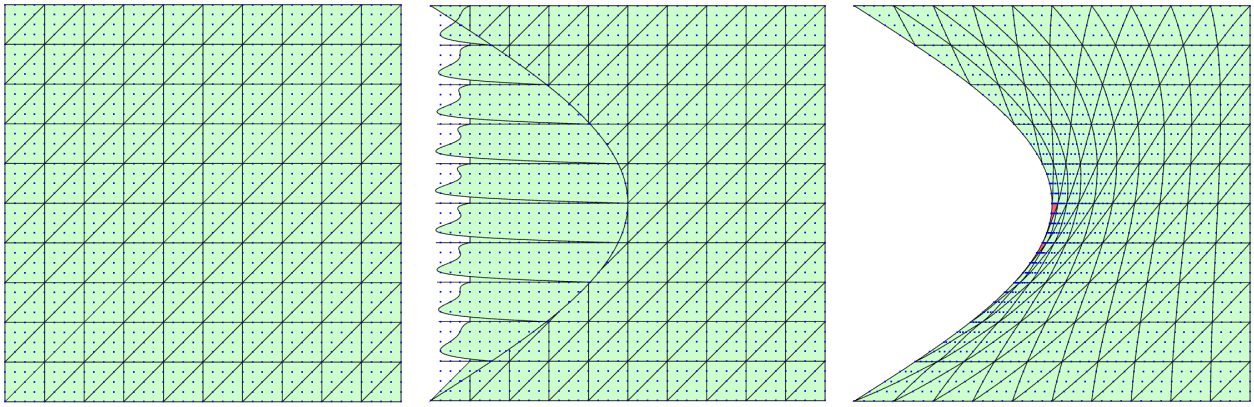
then interpolate via the standard shape functions. The resulting $W_h \in V_h^p$ is C^0, so $\tilde{g}^{ij} = W_h g^{ij} W_h^T$ inherits *only* the jumps already present in g^{ij}, which the $\tilde{\alpha}$-projection handles as before. The averaging itself is cheap: it is a lumped-mass L^2 projection onto V_h^p, equivalent to area-weighted nodal averaging. A full (non-lumped) L^2 projection would require an additional global mass-matrix solve but is not necessary, since the Winslow-like operator is relatively insensitive to moderate blur in the target metric.

1.4 Numerical Results/Tests

We first run the unskewed `w2.m` setup of Section 0.1 through the W-modified solver (`w2_W.m`). The reference mesh, boundary perturbation, and polynomial order are identical to `w2.m`; only g^{ij} is replaced by $\tilde{g}^{ij} = W_h \, g^{ij} \, W_h^T$, with W_h obtained by full (consistent) L^2 projection of the equilateral-target W_K onto V_h^p—the variant closest to the original 2016 formulation. The result is in Figure 5, to be compared directly to Figure 1.

The smoothed mesh has $\min J = -4.4 \times 10^{-4}$: two interior elements are slightly inverted (highlighted in red in Figure 5(c)), where `w2.m` produced $\min J = +3.1 \times 10^{-3}$ and all elements

valid. A regression test (`w_regression.m`) verifies that $W_h \equiv I$ reproduces the unmodified solver bit-for-bit, so the inversion is from the formulation, not the implementation.



(a) Reference mesh. (b) Initial configuration. (c) After W-modified Winslow

Figure 5: `w2_W`: the `w2.m` setup run through the W-modified Picard solver with W_h constructed by consistent L^2 projection onto V_h^p. (a) and (b) are identical to their counterparts in Figure 1. In (c) the two red elements (centroids $(0.46, 0.37)$ and $(0.50, 0.47)$) have $\min J < 0$, so the W-modified output is inverted on a problem where the original 2016 formulation produced a valid mesh.

Running the same modification on the skewed reference (`w2_skewed_W.m`). Applying the same W-modified solver to the `w2_skewed` setup of Section 0.1—the sliver-cluster reference—produces qualitatively different behavior: the Picard iteration does not converge. After an initial residual of ≈ 5, successive iterates oscillate in the range 0.4–5 for roughly thirty steps, then amplify to residuals of 20–90, and finally hit the 100-iteration cap with no sign of settling. The unmodified solver, by contrast, converges cleanly on the same setup in 14 iterations (Figure 2). The divergence is not an implementation artifact; `w_regression.m` continues to pass.

This is the failure mode anticipated qualitatively in Section 1.3, where we noted that the jumps of W_K (or, more generally, the gradient of W_h) are *external* to the iterate and do not decay with Picard, unlike the jumps of g^{ij} which shrink as $\partial x_h/\partial\xi$ smooths. The product-rule expansion of $\tilde{\alpha}_j = -\partial_i \tilde{g}^{ij}$ makes this concrete. With $\tilde{g}^{ij} = W_h^{ik} g^{kl} W_h^{jl}$,

$$\tilde{\alpha}_j \;=\; -\,W_h^{ik} W_h^{jl}\,\partial_i g^{kl} \;-\; 2\,(\partial_i W_h)\cdot g \cdot W_h. \tag{10}$$

The first term is the original α conjugated by W_h. Like the unmodified α, it is driven by the *inter-element jumps* of $\partial x/\partial\xi$ — not its magnitude, which does not shrink, but the discontinuities of the per-element gradient across element interfaces. (These exist because $x_h \in V_h^p$ is C^0 but not C^1, so $\partial x_h/\partial\xi$ is single-valued inside each element but jumps across the boundary between adjacent elements.) These jumps shrink as x_h relaxes toward its fixed point, decaying toward a small discretization-error floor controlled by p and h (the converged x_h is the best polynomial approximation in V_h^p of a genuinely smooth harmonic map, so the residual gradient mismatch at interfaces reflects only the polynomial truncation error, not roughness in the iterate itself). They do not reach zero at finite p.

The second term is *entirely new* and is independent of the current iterate: W_h is fixed once by the reference geometry, so $\partial_i W_h$ never decays with Picard. When the reference mesh is uniform, W_h is a constant field, $\partial_i W_h = 0$, and this new term vanishes—which is why `w2_W.m` converges

and only exhibits a mild fixed-point inconsistency (the two inverted interior elements) rather than an iteration-level failure. When the reference contains a sliver patch, W_h varies sharply across a handful of elements and $\partial_i W_h$ is locally large; the new term contributes a persistent, iteration-independent forcing to $\tilde{\alpha}$ that neither the V_h^p projection nor the Picard loop can absorb. Two failures result:

- *The operator is pushed into a bad regime.* $\tilde{\alpha}_j$ is the advection coefficient in the x-equation $\partial_i(\tilde{g}^{ij}\partial_j x_k) + \tilde{\alpha}_j\,\partial_j x_k = 0$. A persistent large $\tilde{\alpha}$ makes the operator effectively advection-dominated, which standard continuous Galerkin handles badly without stabilization (SUPG, upwinding, etc.).

- *The Picard loop loses its contraction.* Picard converges when the per-step projection error sits below the iteration's natural contraction rate. The relevant quantity is not the magnitude of α (which does not go to zero even in the original method — the converged α is typically a smooth nonzero field), but its *spatial roughness at the mesh scale*: the inter-element discontinuities of α that the L^2 projection has to absorb. In the original formulation this roughness is controlled by the inter-element jumps of $\partial_i g^{ij}$, which decay alongside the iterate to a small discretization-error floor. With the new term, the discontinuities of $\partial_i W_h \cdot g \cdot W_h$ are pinned at the value set by the reference geometry and never decay. The projection still runs, but the per-step roughness it has to absorb is constant; once that constant exceeds the contraction-rate budget the iteration diverges.

The failure in `w2_skewed_W.m` is therefore not a quirk of our particular discretization choice but a consequence of the W-modification itself once $\partial W_h \neq 0$.

This observation is important on its own, independent of any specific cure, because it explains why the scheme as literally written (following the Persson/Gemini exchange) cannot simply be "used". The iteration's self-limiting coefficient smoothing, which is what makes the 2016 formulation so robust, is broken by the W-modification in exactly the regime the modification is supposed to help. Before attempting to stabilize the iteration, it is worth understanding this.

1.5 Summary

The W-modification of Section 1, taken literally as the product-rule substitution $g^{ij} \mapsto W_h g^{ij} W_h^T$ in the 2016 split-form iteration, does not inherit the empirical robustness of the original formulation. Two reasons:

1. *Picard's self-limiting smoothing is broken.* The product-rule expansion of $\tilde{\alpha}_j = -\partial_i \tilde{g}^{ij}$ introduces a new term $-2\,(\partial_i W_h) \cdot g \cdot W_h$ that is independent of the current iterate. It persists as a smooth forcing in $\tilde{\alpha}$ that the L^2 projection cannot absorb and that Picard cannot relax, because it is not driven by $\partial x_h/\partial \xi$. Wherever W_h varies in space—which is exactly where the modification is supposed to do useful work—this term makes the x-equation advection-dominated and Picard non-contractive. Empirically, a single moved interior vertex in the `w2_skewed` reference is enough to send the iteration into oscillatory divergence. Smoothing W_h via lumped or consistent L^2 projection onto V_h^p reduces the magnitude of ∂W_h locally but does not eliminate it—projection trades interface spikes for interior gradients of comparable integrated magnitude. The only way to set $\partial W_h = 0$ is to take W_h globally constant, which defeats the purpose of the modification.

2. *There is no energy whose Euler–Lagrange equation is the modified split form.* A gradient of a scalar functional in x cannot have a non-zero iterate-independent piece, and the ∂W_h term is

exactly that. This rules out any natural variational interpretation in x: there is no Lyapunov function to certify convergence, no minimum principle to guarantee non-inversion at the fixed point, and no standard machinery (line search, backtracking, Newton with globalisation) to stabilize the iteration. We are left with a heuristic fixed-point scheme without convergence theory.

The two observations are not independent: it is because the scheme is not a gradient that the ∂W_h term can appear at all. What makes the 2016 formulation's Picard so robust is precisely the self-consistency condition $\alpha_j = -\partial_i g^{ij}$, which makes α a computed consequence of the iterate; the W-modification breaks this by forcing $\tilde{\alpha}$ to absorb an external contribution. No amount of adjusting the W_h projection or the lumping recovers this structure. The modification is attractive on paper but the iteration machinery it was slotted into does not tolerate it.

2 Reference-mesh preprocessing via Tutte's embedding

Motivated by the negative findings of Section 1, we take a different approach. Rather than modifying the Fortunato–Persson iteration to correct for a poor reference mesh, we commit to using the 2016 scheme *as a black box* and instead construct a well-shaped reference mesh with the same topology.

The whole chain is: given a poorly shaped, possibly tangled high-order 2D mesh, we produce a straight-sided reference mesh with the *same connectivity* whose element shapes are as close to ideal as the topology permits, and then we feed that reference into the 2016 framework with the original mesh's boundary as Dirichlet data.

The preprocessing chain has four steps, each taken up in a subsection below:

1. **Extract the topology** (Section 2.1). Discard the positions of the input mesh and keep only its combinatorial structure: the vertex list, the triangle list, and the identification of boundary vertices.

2. **Tutte's barycentric embedding** (Section 2.2). Map the boundary vertices to a convex polygon (e.g. the unit square), then solve Tutte's linear system for the interior vertex positions. The output is a straight-sided planar mesh with the input topology, guaranteed to have no inverted or degenerate elements.

3. **Mesh optimization** (Section 2.3). *The key observation is that this step acts on a straight-sided, untangled mesh.* The Tutte output of Step 2 is guaranteed to have straight edges and positive element Jacobians, so standard mesh-quality optimization—which is notoriously ill-posed on tangled or curved input—can be applied cleanly. We minimize a shape functional (e.g. the Liu–Joe shape distortion η (5)) while keeping the topology fixed and the vertex positions valid. This turns the "valid but possibly ugly" Tutte output into a "valid and well-shaped" straight-sided reference candidate.

4. **Promotion to high order** (Section 2.4). Apply `nodealloc` to allocate the V_h^p nodes on the straight edges of the optimized mesh, producing a valid high-order reference mesh that matches the input's polynomial degree.

The resulting reference mesh is then used as the input to `elliptic_smoothing.m` with the original (possibly tangled) mesh's boundary node positions as Dirichlet data. The 2016 scheme proceeds unmodified from that point.

2.1 Extracting the topology

Given the input high-order mesh we discard its geometric data—both the linear vertex positions and the high-order node positions—and retain only its combinatorial structure: the list of triangles (triples of linear-vertex indices) and the identification of which edges lie on the exterior boundary. This is enough to run Tutte's algorithm, which operates on the abstract planar graph of the mesh. Whether the input positions were tangled, inverted, badly shaped, or perfectly valid is irrelevant at this stage: the data Tutte sees is the same list of vertices and the same list of triangle connectivities.

Two structural assumptions on the input are required. First, the mesh must be topologically a disk—simply connected with a single exterior boundary. Second, its linear-vertex graph must be 3-connected. Both are satisfied by any standard triangulation of a simply connected planar domain with more than three vertices. Given these, the boundary forms a single cyclic loop of linear vertices, which we extract by walking the edges whose faces have no neighbor in the triangle-to-triangle connectivity. All remaining vertices are classified as interior.

2.2 Tutte's barycentric embedding

Tutte's theorem (1963): if the boundary vertices of a 3-connected planar graph are pinned to a convex polygon and each interior vertex is placed at the centroid of its graph neighbors, then the resulting straight-line drawing is a valid planar embedding—no triangle is inverted or degenerate. The theorem is striking in that it produces a globally valid mesh from purely local barycentric conditions, with no optimization loop, line search, or sign-of-Jacobian tracking.

Given a chosen convex target polygon P (the computational domain we want our reference mesh to live in), the N boundary vertices extracted in Section 2.1 are distributed monotonically around ∂P by **proportional arc length**: if two consecutive boundary vertices are at arc-length distance ℓ on the physical boundary (out of a total physical perimeter L), they are placed at arc-length distance $|\partial P| \cdot \ell / L$ on the target boundary. In other words, boundary vertex spacing on ∂P mirrors the spacing on the physical boundary in proportion. This preserves the *local density* of boundary vertices (vertices per unit arc length) under the map: dense regions on the physical stay dense on the target, sparse regions stay sparse. What is *not* preserved is the 2D area backing each boundary segment; we return to the consequences of that mismatch when we discuss clumping in the starfish example (Section 2.5.4).

Each interior vertex $\mathbf{x}_i$ is then required to satisfy

$$\mathbf{x}_i \;=\; \frac{1}{\deg(i)} \sum_{j \sim i} \mathbf{x}_j, \qquad i \in \text{interior}, \tag{11}$$

where the sum is over the linear-vertex neighbors j of i and $\deg(i)$ is the valence. Collecting one such equation per interior vertex and moving the known boundary contributions to the right-hand side gives the sparse linear system

$$L_{\text{ii}}\, \mathbf{x}_{\text{int}} \;=\; -\, L_{\text{ib}}\, \mathbf{x}_{\text{bnd}}, \tag{12}$$

where $L = D - A$ is the graph Laplacian of the linear-vertex connectivity (D diagonal of valences, A the adjacency matrix), L_{ii} is its restriction to interior–interior entries, and L_{ib} its restriction to interior–boundary entries. The system is symmetric positive definite, and each of the two spatial components of $\mathbf{x}$ is solved independently with a single sparse direct call.

A few practical notes:

- **Boundary parameterization.** We use proportional-arc-length placement as described above. Any monotone bijection from the input boundary loop onto ∂P would produce a valid Tutte embedding; the proportional-arc choice is what keeps the map's local information content tied to the physical—it gives the clumping phenomenon a clean diagnostic interpretation instead of tangling it with an arbitrary boundary-placement choice. Uniform-arc (every boundary vertex equally spaced on ∂P, regardless of physical) is a simpler alternative but loses that correspondence.

- Tutte's output is uniquely determined by the topology and the boundary map. Distinct topologies produce distinct embeddings; distinct boundary parameterisations of the same topology produce distinct embeddings.

- The output is valid in the sense that every triangle has positive area, but it is not optimized for element shape. Vertices with high or irregular valence, or whose graph neighborhood is highly asymmetric, typically produce short-edge or long-edge triangles. Improving those shapes is the job of Section 2.3.

2.3 Shape optimization of the straight-sided mesh

The Tutte output of Section 2.2 is a valid straight-sided triangulation of the target polygon P: boundary vertices are pinned to ∂P, interior vertices sit at Laplacian positions, every element has positive area. We now relocate the interior vertices to improve element shapes. Two features of this step make it tractable: the input is untangled (every element has positive area) and straight-sided (every Jacobian J_K is a single 2×2 matrix per element). Both of these fail for the original input mesh—which is why this optimization is deferred to after Tutte.

On each element K, the Liu–Joe shape distortion of Equation (5) reduces to

$$\eta_K \;=\; \frac{\|J_K\|_F^2}{d\,|\det J_K|^{2/d}}, \qquad \eta_K \geq 1, \tag{13}$$

where J_K is the Jacobian of the affine map from the ideal (equilateral) simplex to K. For the present linear problem J_K is an explicit, closed-form function of the three corner positions of K.

We aggregate these per-element values into a single scalar objective over the interior-vertex positions $\mathbf{x}$:

$$F(\mathbf{x}) \;=\; \sum_{K \in \mathcal{T}_h} \eta_K(\mathbf{x})^2. \tag{14}$$

Squaring, rather than taking the plain sum, has two practical effects. First, it penalizes outlier elements quadratically, so a single near-degenerate element cannot hide inside a large overall sum of moderate values. Second, and more importantly, η_K diverges as $|\det J_K| \to 0$; the squared objective inherits this divergence and acts as an effective infinite barrier against element inversion. Because the Tutte output in Section 2.2 gives a strictly positive Jacobian at every element, we start inside this barrier, and any descent direction that respects F stays inside it.

The problem is a standard unconstrained nonlinear optimization over $\mathbb{R}^{2n_{\text{int}}}$ (two coordinates per interior vertex, boundary vertices held fixed). Gradients are analytic—J_K is affine in the three corner positions, so η_K is a rational function of the interior-vertex coordinates with explicit derivatives—and standard quasi-Newton solvers (L-BFGS, trust-region Newton, BFGS with Armijo line search) converge without special handling. The Tutte output is the initial guess.

This is the straight-sided specialization of the curved-element mesh optimization problem of Shi–Persson [3], whose Equation (4) generalizes η_K to high-order elements by integrating a pointwise

distortion over each curved element. In the present pipeline we do not need that generalization because the mesh entering the optimization step is straight-sided by construction.

2.4 Promotion to high-order reference

The output of Section 2.3 is a straight-sided linear triangulation of the target polygon P: fixed topology inherited from the input, boundary vertices pinned, interior vertices placed at a minimizer of the discrete shape objective. Nothing about the high-order structure has been touched yet.

Promoting to high order is trivial: run `nodealloc(msh, porder)` with the same polynomial order as the input mesh. The routine allocates high-order DG nodes by interpolating the shape functions on each straight-sided element from its three linear corner positions. Because the linear triangulation is straight-sided, there is no ambiguity: high-order nodes sit exactly where the element's barycentric interpolation places them, i.e., along straight edges and on a regular pattern in the interior of each element. The resulting high-order mesh inherits the element quality of the linear mesh from Section 2.3: every J_K of the reference element is still the same 2×2 affine Jacobian, and the Liu–Joe distortion η_K at each element is the same value it had at the end of the shape optimization.

With this reference mesh in hand, we hand it to `elliptic_smoothing.m`, the original 2016 Fortunato–Persson solver, with the *original* input mesh's boundary node positions as Dirichlet data. No further modification is needed—the smoother runs in exactly the regime it was designed for: a well-shaped straight-sided reference with a prescribed (possibly curved) physical boundary.

2.5 Numerical examples

2.5.1 Example 1: regular grid, `w2.m` topology

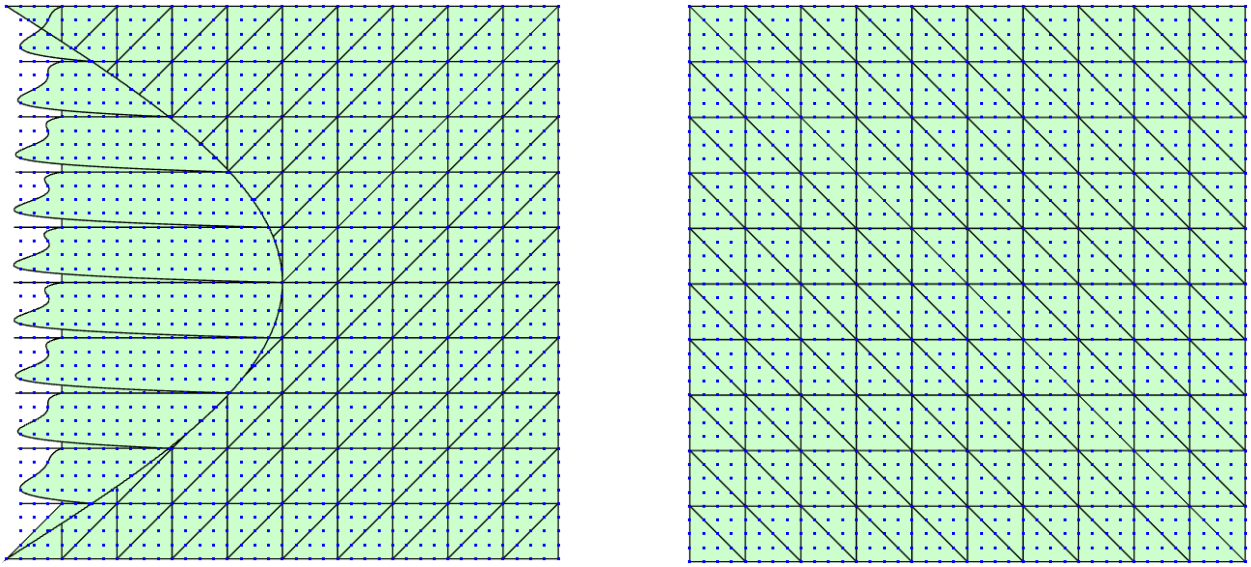
We take the tangled input from `w2.m` (Figure 1, (b)) and feed it through the pipeline with no other information. In particular, the pipeline is *not* told that the input came from `mshsquare`; it sees a high-order mesh whose boundary is the unit square on three sides and the curve $x = 1 + 0.5\sin(\pi y)$ on side 4, with heavily tangled interior high-order nodes.

Figure 6(a) is the input. Tutte with target $= [0, 1]^2$ produces (b)—the uniform 11×11 grid of `mshsquare(11, 11)`, recovered from topology alone, with every element at $\eta_K = 3/2$ (the Liu–Joe distortion of a right-isoceles triangle). Running BFGS on $F = \sum_K \eta_K^2$ with the equilateral simplex as the ideal element does not budge a node: the uniform grid is already a local minimum, $|\nabla F| = 4.6 \times 10^{-12}$ at iteration 0. Feeding (b) into `elliptic_smoothing.m` as the reference mesh, with (a)'s boundary as Dirichlet data, then reproduces the `w2.m` output of Figure 1(c).

2.5.2 Example 2: unstructured Delaunay on the unit square

Incrementally harder: the same domain as Example 1 but with an unstructured topology, so the Tutte Laplacian becomes non-trivial and the shape optimizer has real work to do. The input is constructed in two steps shown in Figure 7: a clean Delaunay triangulation of the unit square with 40 boundary samples and 60 interior random points (a), then a mild tangling that perturbs each interior linear vertex by a uniform random displacement of amplitude 0.1 and each interior high-order DG node by amplitude 0.03 (b). Boundary linear and high-order nodes are left alone.

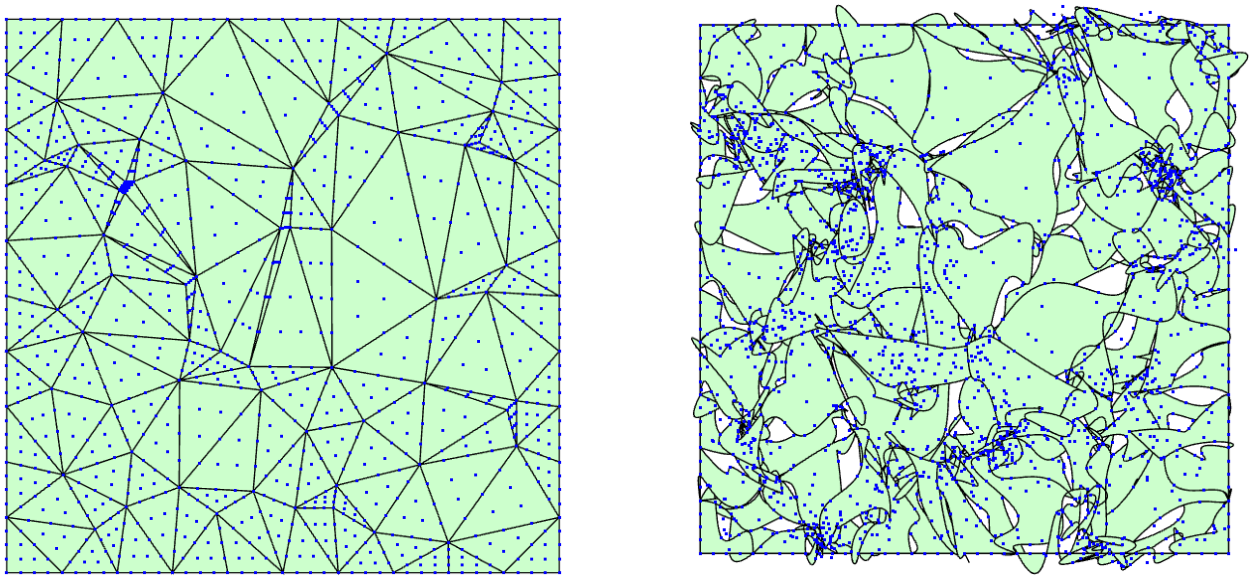
The pipeline proceeds through the four stages of Figure 8. Tutte (b) untangles the interior with a single sparse solve ($\max \eta = 2.64$, no inversions), placing each interior vertex at the centroid of its graph neighbors. Shape optimization (c) redistributes the interior vertices with 137 BFGS iterations, lowering $\max \eta$ from 2.64 to 2.30 and F from 198.2 to 195.5; unlike Example 1, the



(a) Input: tangled mesh from `w2.m`.

(b) After Tutte with target $= [0,1]^2$.

Figure 6: `w2_tutte.m`: the pipeline applied to `w2.m`'s tangled input. (a) is identical to Figure 1(b), shown here for reference. (b) is the Tutte output.

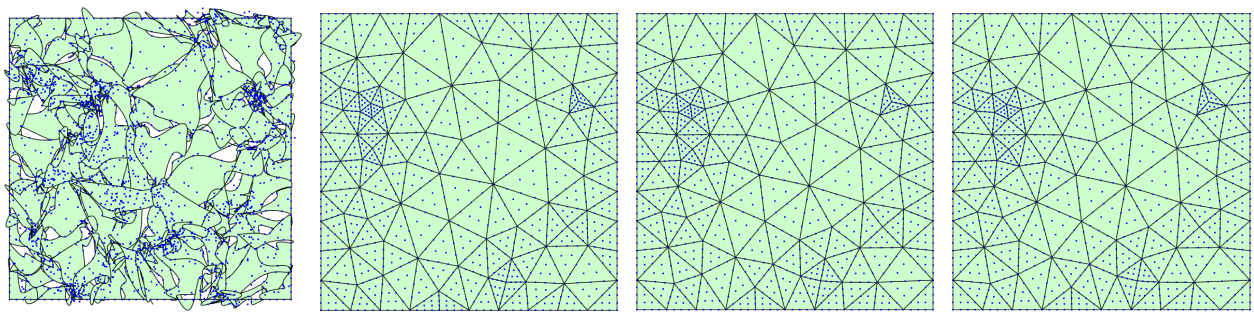


(a) Clean random Delaunay.

(b) After mild interior tangling.

Figure 7: Construction of the Example 2 input: a clean random Delaunay triangulation of $[0,1]^2$ (a) has its interior linear and high-order nodes mildly perturbed to produce the tangled input of the pipeline (b).

optimizer has real work to do on an unstructured topology, though the two sliver fans at high-valence vertices (visible on the left and right of c) cannot be removed without retriangulation and remain as the worst elements. Handing the optimized reference to `elliptic_smoothing.m` with the tangled input's boundary as Dirichlet data (d) is trivial here—because the input's boundary was left unperturbed, it matches the reference boundary exactly, and Winslow converges in a single iteration reproducing (c).



(a) Tangled input. (b) Tutte of (a). (c) Shape-opt of (b). (d) Winslow of (a) w.r.t. (c).

Figure 8: Example 2 pipeline. (a) is the tangled input of Figure 7(b). (b) is the Tutte output. (c) is the output of shape optimization on the Tutte output. (d) is the result of Fortunato–Persson Winslow smoothing with (c) as the reference mesh and (a)'s boundary as Dirichlet data; since the input boundary was left unperturbed, (d) is identical to (c).

2.5.3 Example 3: convex stadium

Next up: a convex domain with a non-trivial mixed smooth-curved / straight boundary. The stadium is a rectangle of length $2a$ capped with a half-disk of radius b at each end; the signed distance function is

$$f_d(\mathbf{p}) \;=\; \sqrt{\max(|x| - a,\, 0)^2 + y^2} \;-\; b.$$

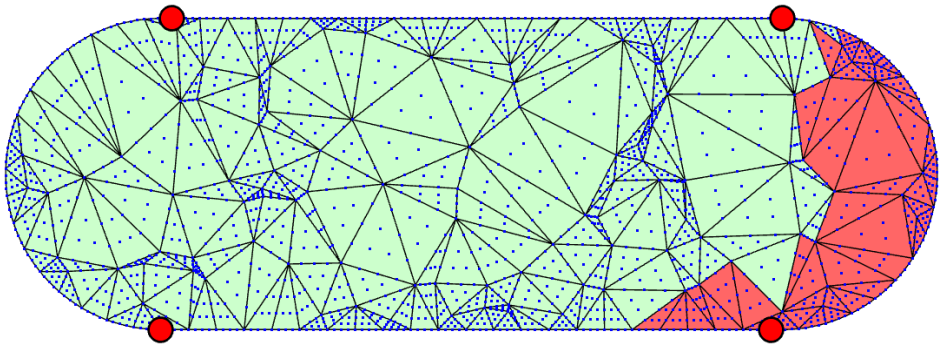
With $a = 1$ and $b = \frac{1}{2}$ the domain has aspect ratio $3:1$—long enough that Tutte's map to the unit disk is genuinely non-trivial, but the boundary is convex and smooth so the pipeline should work cleanly.

Quantitative comparison of (c) and (e), with cost. The two Winslow outputs differ by a consistent margin in favor of the shape-opt path (Table 1). On the 278-element stadium mesh, shape-opt reduces mean η^2 by $\sim 26\%$ and worst-element η by $\sim 32\%$, at the cost of one extra $\sim 0.4\,\mathrm{s}$ stage—less wall clock than a single Winslow solve. The full with-shape-opt path costs roughly $2\times$ the no-shape-opt path. Shape-opt is not a heavy step; whether to include it is a routine cost-accuracy tradeoff, not a pipeline structure decision.

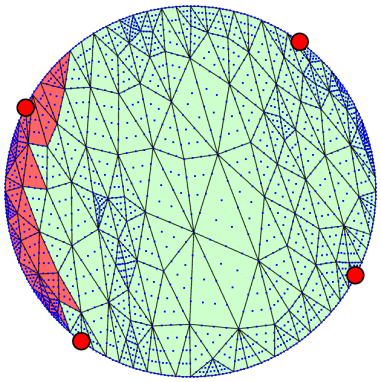
2.5.4 Example 4: smooth non-convex starfish

Next: a smoothly curved, strongly non-convex boundary with no corners. We take a five-armed starfish,

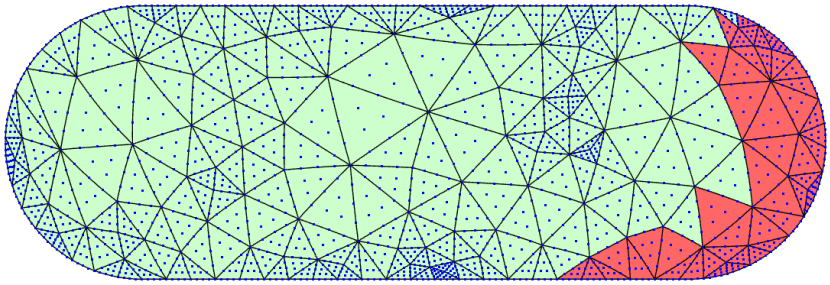
$$r(\theta) \;=\; 1 + 0.40\,\cos(5\theta) + 0.03\,\sin(2\theta), \qquad (x, y) \;=\; r(\theta)\,(\cos\theta, \sin\theta),$$



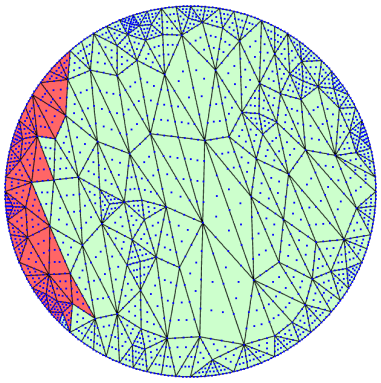
(a) Physical stadium; red is the right end-cap (the Delaunay triangles that will become the left boundary clump on the disk).



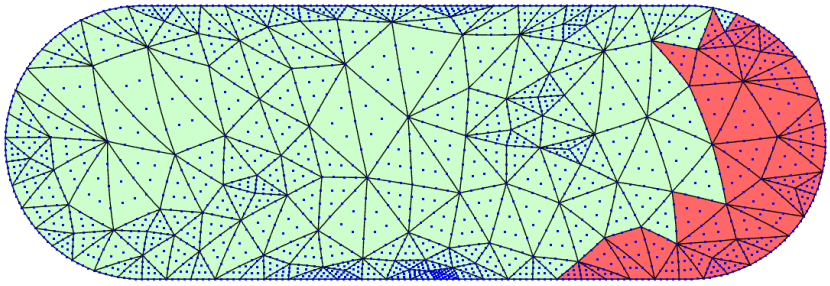
(b) Tutte (disk).



(c) Winslow, direct from Tutte (no shape-opt).



(d) After shape-opt.



(e) Winslow, after shape-opt.

Figure 9: Stadium pipeline with unit-disk target. Red dots in (a) and (b) mark the four curvature-transition points $(\pm a, \pm b)$ where the straight sides meet the semicircles (the stadium boundary is C^1 smooth there, not cornered—these are curvature-discontinuity points that would anchor any sliding parameterization). Row 1: physical stadium with the right end-cap highlighted for tracking. Row 2: no-shape-opt path (Tutte directly into Winslow). Row 3: with-shape-opt path (Tutte $\rightarrow$ shape-opt $\rightarrow$ Winslow). Both paths produce valid, well-shaped stadium meshes and are nearly indistinguishable—on a convex smooth domain Winslow's bounded-distortion map does most of the work and shape-opt is a modest polish.

Table 1: Quality and wall-clock comparison of the two Winslow paths on the stadium, 278 elements. Panel labels refer to Figure 9.

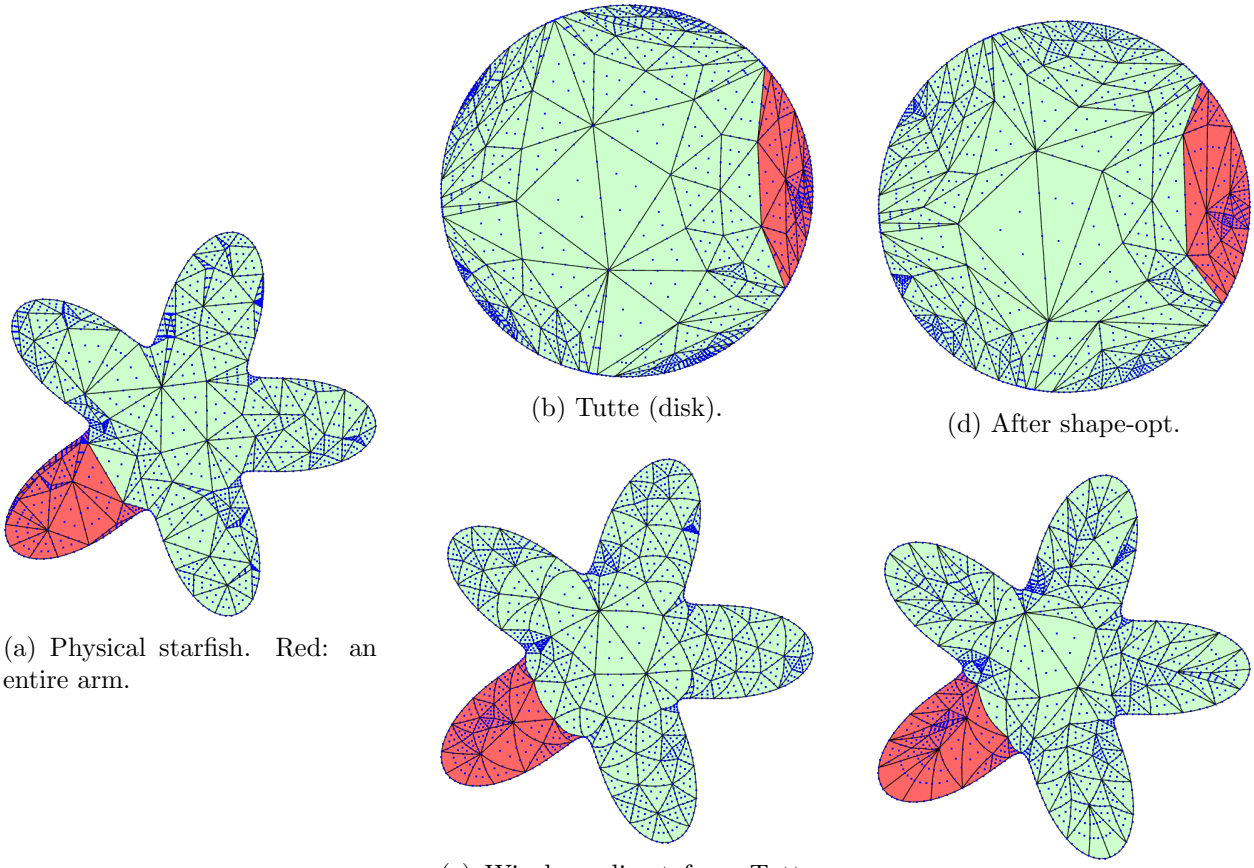
	panel (c) no shape-opt	panel (e) with shape-opt	reduction
mean(η^2)	3.04	2.24	−26%
max(η)	3.89	2.66	−32%
Wall-clock time:			
shape-opt (b)→(d)	—	0.40 s	
Winslow	0.49 s	0.65 s	
total	0.49 s	1.05 s	2.1×

with the principal $\cos(5\theta)$ term producing the five arms and the small $\sin(2\theta)$ modulation breaking exact 5-fold symmetry. The arm-to-bay radial ratio is $\sim 2.3:1$, so the arms are pronounced. We run the pipeline with the unit disk as the Tutte target.

Figure 10 shows the pipeline in two parallel variants: the no-shape-opt path (column 2, Tutte directly into Winslow) and the with-shape-opt path (column 3, Tutte → shape-opt → Winslow). Column 1 is the physical starfish used by both paths. To make the geometric effect legible we highlight the largest connected clump of near-boundary triangles in the bottom-right sector of the Tutte output and trace the same triangle indices (preserved through Tutte and shape optimization, since only vertex positions change) back to the physical mesh.

Clumping on the Tutte disk as a diagnostic. We have now seen two different physical geometries produce structured sliver clumps on the Tutte disk. The stadium (Section 2.5.3, Fig. 9b) produced two clumps, one per semicircular end-cap on the physical. The starfish (Fig. 10b) produces five clumps, one per arm. In both cases the clumps are *localized* to specific arcs of the disk and the top/bottom/body portions of the disk are comparatively clump-free. A unifying interpretation:

- Under proportional-arc Tutte (Section 2.2), the boundary-vertex density on the reference arc matches that on the corresponding physical segment.
- The *interior* triangles attached to that boundary segment are placed by Tutte (Section 2.2). If the physical region backed by that boundary segment has small area per boundary vertex (dense boundary sampling), Tutte ends up placing the adjacent interior vertices *close* to the corresponding short disk arc. The resulting triangle is thin in both the tangential direction (its two boundary vertices are close together on the arc) and the radial direction (the interior vertex is pulled toward the arc)—a boundary-hugging sliver. Many such triangles side-by-side form the clump.
- *What makes boundary sampling locally dense varies by domain.* For the stadium it is *boundary curvature*: Delaunay resolves the curved caps with many small edges while the straight sides use few large edges. For the starfish it is *narrow 2D shape*: each arm has moderate perimeter share but only $\sim 1/5$ of the area, so all of its triangles are boundary-adjacent.
- The resulting diagnostic is the same: each clump on the disk points at a specific physical region where boundary sampling is locally dense relative to interior area. Equivalently, for a convex target P, a physical region gets an angular wedge of P whose opening is set by



(a) Physical starfish. Red: an entire arm.

(b) Tutte (disk).

(d) After shape-opt.

(c) Winslow, direct from Tutte (no shape-opt).

(e) Winslow, after shape-opt.

Figure 10: Starfish pipeline with unit-disk target. The 27 highlighted triangles fill one arm of the physical starfish (a). Column 2 is the *no-shape-opt path*: Tutte lays the arm's boundary vertices onto a thin angular wedge of the disk (b), producing an inward sliver fan, and Winslow transports that fan back to the physical arm (c) where it unwinds into a decent radial fan of moderate-aspect-ratio triangles. Column 3 is the *with-shape-opt path*: shape optimization (d) compresses the wedge slightly but cannot reshape its sliver structure with the boundary pinned; Winslow then transports the polished wedge back to a similar radial fan (e). Shape-opt's contribution is a modest improvement of the worst element on the physical mesh (min $|J|$ $0.0011 \rightarrow 0.0026$), not a qualitative change; Winslow does most of the work.

its perimeter share, not its area share; when the two shares disagree (elongated protrusions, tightly-sampled curves), the triangles inside are compressed tangentially into slivers on the disk. The mechanism causing the mismatch differs by domain; the pipeline handles both the same way.

- What Winslow does about it: it maps each slivered wedge back to its narrow physical region, compressing triangles tangentially and stretching them radially, which undoes the Tutte-side distortion. For smooth physical boundaries this graceful transport recovers moderate aspect ratios—see Fig. 10(e) for the starfish and Fig. 9(e) for the stadium. It is not guaranteed in general; for non-smooth physical boundaries the inverse map can fail to fully unwind the compression, as we will see in Example 2.5.5.

Quantitative comparison of (c) and (e), with cost. The two Winslow outputs differ substantially in favor of the shape-opt path (Table 2). On the 218-element starfish mesh, shape-opt cuts mean η^2 by $\sim 49\%$ and worst-element η by $\sim 55\%$, at the cost of a $\sim 0.2\,\mathrm{s}$ shape-opt stage. The full with-shape-opt path costs roughly $1.8\times$ the no-shape-opt path. Unlike the stadium (a convex smooth domain, where Winslow alone was already excellent), a non-convex physical with narrow arms gives shape-opt enough room that skipping it leaves visible quality on the table.

Table 2: Quality and wall-clock comparison of the two Winslow paths on the starfish, 218 elements. Panel labels refer to Figure 10.

	panel (c) no shape-opt	panel (e) with shape-opt	reduction
mean(η^2)	6.84	3.51	-49%
max(η)	10.46	4.70	-55%
Wall-clock time:			
shape-opt (b)→(d)	—	0.21 s	
Winslow	0.66 s	0.98 s	
total	0.66 s	1.19 s	$1.8\times$

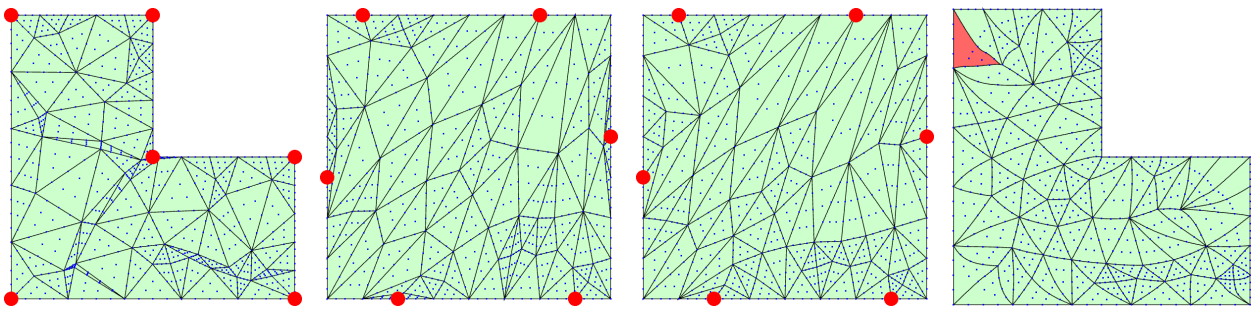
Aside: what shape optimization actually optimizes. Shape-opt minimizes $\sum_K \eta_K^2$, which measures *shape* quality, not element *size*. In the starfish comparison (Fig. 10) the no-shape-opt Winslow output can look more uniform at a glance because its triangles are more similarly sized, but the shape-opt output has lower distortion by every η-based metric (see Table 2). Visual size-uniformity and shape quality are different objectives.

2.5.5 Example 5: non-convex L-shape, two target shapes

The next incremental step is a non-convex polygonal domain. We take the L-shape (unit square minus $[0.5, 1] \times [0.5, 1]$), which introduces a reflex corner at $(0.5, 0.5)$ and six boundary corners total. The input is a constrained Delaunay of the L with 30 boundary samples and 41 interior points (panel (a) of Figures 11 and 12).

Because the L-shape is non-convex, the Tutte target P must be a convex polygon containing it topologically—some convex region we will use as the computational domain before handing off to Fortunato–Persson. We run the pipeline with two natural choices of P: the unit square and the unit disk.

Unit-square target. Figure 11 shows the four stages. Tutte with $P = [0, 1]^2$ produces a valid straight-sided reference (b, $\max \eta = 7.35$) on which shape optimization does real work, pulling $\max \eta$ down to 4.15 over $O(10^2)$ BFGS iterations (c). Feeding this reference to `elliptic_smoothing.m` with the L-shape boundary as Dirichlet data (d) produces a mesh of the physical L-shape, but with $\min I = -0.019$ (one element slightly inverted, shown in red) and $\max \eta = 10.84$. The failure is geometric. For a polygonal target, Tutte has to pin some boundary vertices to the target's corners, otherwise the boundary it embeds onto is a corner-chopped polygon inscribed in the target rather than the target itself. The square-target code picks four boundary vertices by uniform index spacing around the boundary loop and pins them to the four square corners, independent of which vertices are L corners. For this mesh none of the L's six corners sit at the four chosen indices, so all six end up on the interior of straight square edges (red markers in b). When Winslow maps the reference back to the L-shape, each of those six vertices has to develop a sharp (or reflex) corner out of a straight reference neighborhood, and the worst case produces the inverted element in d.

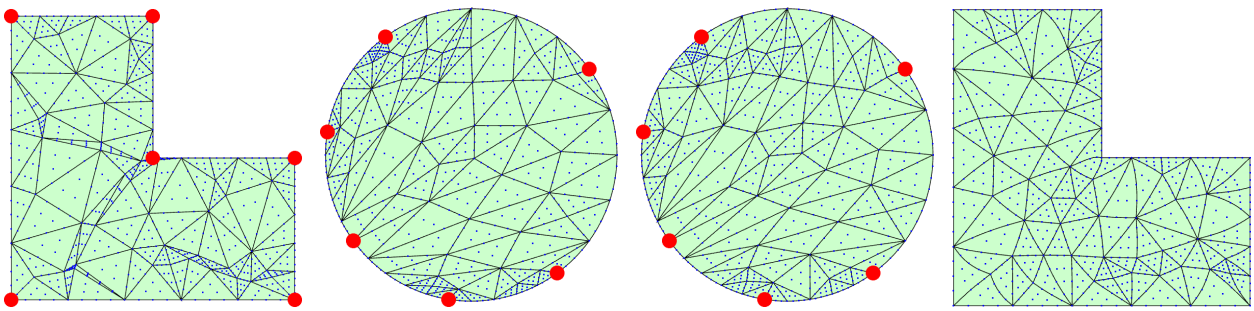


(a) Clean input. (b) Tutte (square). (c) Shape-opt. (d) Winslow.

Figure 11: L-shape pipeline with unit-square target. The six L-corners are marked in red; Tutte (b) places all six on the interior of straight square edges, because the boundary vertices it pins to the four square corners happen to be non-corner samples of the L. The Winslow step (d) produces a valid-looking L-shape mesh but with one slightly inverted element ($\min I = -0.019$, shown in red) near one of those six vertices that had to develop a sharp corner from a straight reference neighborhood.

Unit-disk target. Figure 12 repeats the pipeline with P the unit disk. Tutte distributes the L's boundary uniformly around the circle (b), shape optimization settles to $\max \eta = 4.38$ over 113 BFGS iterations (c), and Winslow produces a clean L-shape mesh (d) with $\min I = 0.17$ and $\max \eta = 4.29$—all elements valid, and $2.5\times$ better in $\max \eta$ than the square-target output. The disk has no corners, so no boundary vertex is artificially assigned corner or non-corner status; the L's six corners are free to sit wherever they land on ∂P and Winslow reshapes them into sharp L corners without fighting the reference.

Aside: what happens if we let boundary nodes slide? The pipeline as described above keeps boundary vertices pinned at two places: during shape-opt (boundary vertices stay at the Tutte-assigned disk positions) and during Winslow (boundary vertices stay at the physical L-shape positions, connected to the reference boundary by index). A natural question is whether *sliding* boundary vertices along ∂P or $\partial \Omega$—pinning only the L's six corners, letting every other boundary vertex move along its assigned edge of the disk or the L—could produce a better physical mesh. Crossing the two possibilities at each stage gives four configurations:



(a) Clean input. (b) Tutte (circle). (c) Shape-opt. (d) Winslow.

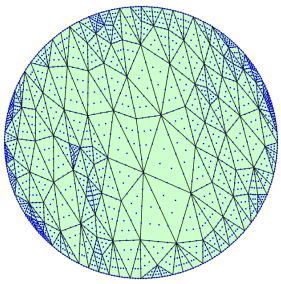
Figure 12: L-shape pipeline with unit-disk target. The six L-corners are marked in red; proportional-arc Tutte (b) places them around the circle at positions proportional to their L-perimeter arc-length. No artificial corner alignment; the Winslow output (d) is valid ($\min I = 0.17$, $\max \eta = 4.29$) and more than 2.5× better in $\max \eta$ than the square-target output of Figure 11.

- **A. Pinned shape-opt, pinned Winslow** (*baseline*). The pipeline used throughout this section; the output is Figure 12(d).
- **B. Sliding shape-opt, pinned Winslow**. Let shape-opt redistribute non-corner boundary vertices along the disk (with the six L-corners pinned on the circle at their proportional-arc positions). Derive new physical boundary positions from the slid disk positions via proportional arc length along $\partial\Omega$, and run a pinned Winslow with that new physical boundary as Dirichlet data.
- **C. Pinned shape-opt, sliding Winslow**. Keep the reference exactly as in (A); let Winslow's PDE solve itself slide the physical boundary nodes along $\partial\Omega$, with only the six L-corners pinned. No redistribution happens on the reference side at all—boundary-vertex density on the disk stays as proportional-arc Tutte laid it down.
- **D. Sliding shape-opt, sliding Winslow**. Let both stages slide: shape-opt adjusts the disk distribution and Winslow adjusts the physical distribution.

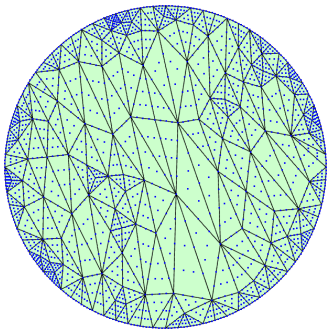
The motivation is that pinning enforces a specific boundary density at each stage, and if those densities are suboptimal, a sliding stage could in principle lower $\sum \eta^2$ further. Whether the improvement on one side translates to the other is an empirical question.

We tried configuration (B) on two test cases: first the convex stadium of Section 2.5.3, then the L-shape of the present section. The contrast is sharp.

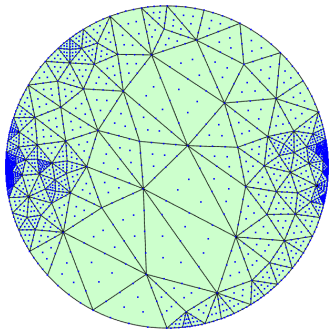
Stadium ((B) helps). On the smooth convex stadium, (B) lets shape-opt slide the disk's boundary between a small set of anchor pins; the slid distribution (Figure 13c) lowers $\sum \eta^2$ on the disk relative to (A), and the proportional-arc-length pullback transfers that improvement to the physical side cleanly (e). The slid disk piles non-anchor boundary vertices into a few angular gaps—two visible clumps of small reference triangles near the left and right ends of the disk—and the pullback produces correspondingly dense regions of small physical elements near the curved end caps. Crucially, those small physical elements remain *well-shaped*: the stadium has no concave or convex corners for the dense regions to compress into, so aggregate distortion improves over (A) on every metric.



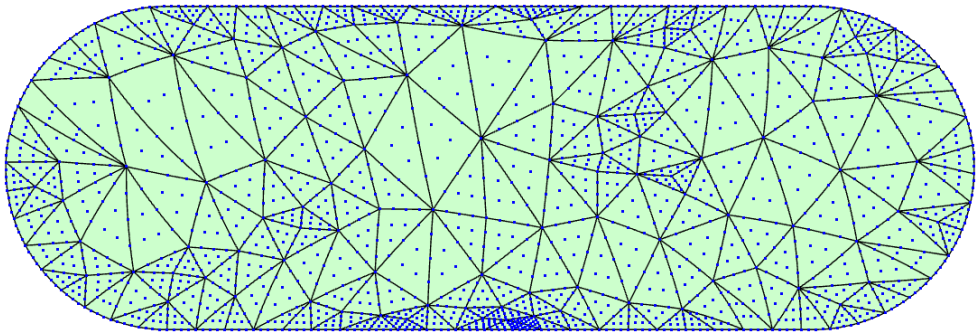
(a) Tutte (disk).



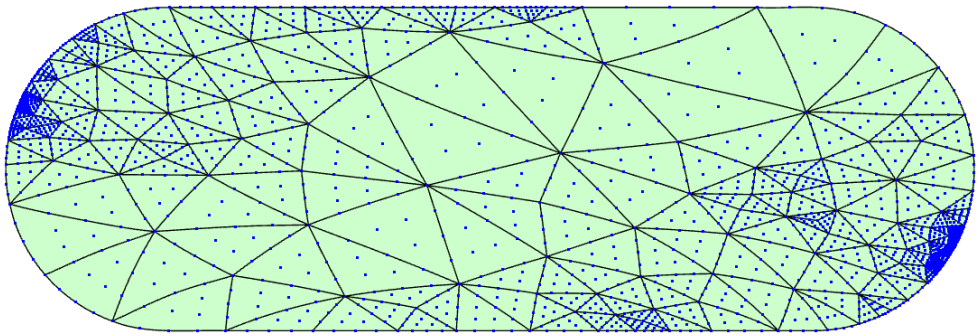
(b) (A) pinned shape-opt.



(c) (B) slid shape-opt.



(d) (A) pinned Winslow.



(e) (B) slid-disk Winslow.

Figure 13: Configurations (A) and (B) on the convex stadium of Section 2.5.3. Row 1: the proportional-arc Tutte disk (a), shared input to both paths. Row 2: shape optimization on the disk—(A) pins every boundary vertex (b); (B) pins eight evenly-spaced anchors and lets the remaining boundary vertices slide along the unit circle (c). Rows 3–4: pinned Winslow on the physical stadium—(A) uses the original stadium boundary as Dirichlet data (d); (B) uses proportional-arc-pulled-back stadium positions derived from the slid disk (e). The slid-disk redistribution piles boundary density toward the curved end caps, but the smooth convex geometry has no corner for that density to compress into, so element shape in (B) stays good across the whole mesh.

L-shape ((B) fails). On the L-shape with the unit-disk target the same disk-side freedom backfires. The slid disk (Figure 14c) does what shape-opt is told to do: it lowers $\sum \eta^2$ on the disk by piling non-corner boundary vertices into the angular gaps where proportional-arc Tutte left the density coarsest. The visible side effect is a new clump of small reference triangles next to two of the pinned L-corners. (Disk-side clumping is the diagnostic for dense physical-side spacing; the same mechanism applies here, except shape-opt is creating the dense spacing rather than the input dictating it.)

The proportional-arc-length pullback transfers that redistribution to a new physical L-shape boundary, and a pinned Winslow runs with that boundary as Dirichlet data. The output (e) has a clump of near-degenerate elements next to each L-corner that inherited a clumped reference region. Aggregate $\sum \eta^2$ on the physical side goes *up*, not down: the few corner-adjacent elements with extreme distortion swamp the modest improvement everywhere else. On the unit-square target the pullback produces a physical boundary that pinned Winslow fails to untangle at all.

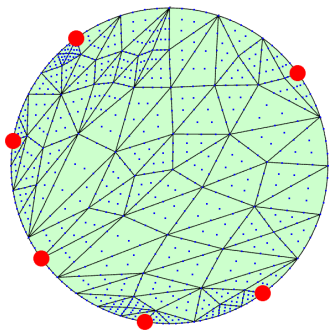
Moral and outlook. The reference optimizer has no information about how disk-side density pulls back to the physical domain; giving it freedom over boundary spacing lets it lower its own objective at the cost of the pullback. The cost is harmless on the smooth convex stadium and catastrophic on the cornered L-shape. (B) is therefore not a universally bad idea—it is a bad idea for any input whose boundary contains a corner that the optimizer can find and exploit. A pinned-shape-opt formulation is the safer default for the L-shape and similar polygonal-corner inputs. The same failure mode will surface again whenever we relax a constraint that was implicitly protecting the physical mesh from reference-local exploitation; we already saw it for shape-opt flips. Configurations (C) and (D) require sliding inside the Winslow solve itself; we have started that implementation but do not yet have stable results to report for the curved-boundary case.

2.5.6 Example 6: comb (cooperative reference shape)

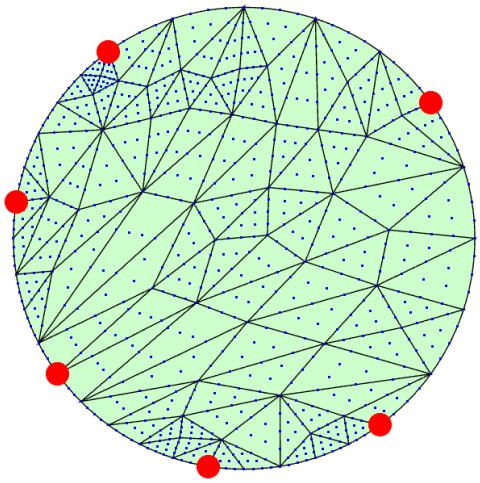
The comb is the first example with both narrow features (three teeth of width 0.18 and height 0.18) and re-entrant corners (the eight corners at the tooth bases). It is built to stress every stage of the pipeline at once: Tutte must spread sixteen sharp boundary corners into a convex target without folding, shape-opt must redistribute elements without exploiting the re-entrant corners, and the Winslow pullback must cross the tooth-base notches without inverting curved elements there.

Geometry and meshing. The comb has a base rectangle $[0, W] \times [0, H_{\text{base}}]$ with $W = 0.92$ and $H_{\text{base}} = 0.20$, capped by three rectangular teeth of width 0.18, height 0.18, gap 0.10, and side margin 0.10. A constrained Delaunay triangulation with ~ 25 boundary samples per unit length and 350 interior samples produces a clean reference mesh. The constrained-Delaunay graph contains 2-cuts at the tooth bases (re-entrant corners with no interior point bridging them); the helper `make_3_connected.m` adds anchor points to restore 3-connectedness before Tutte.

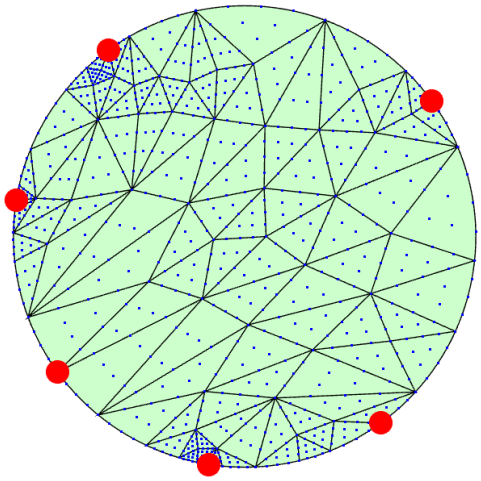
Tutte to a kinder reference: the dome. The unit disk gives each tooth roughly 20° of arc on the target boundary, leading to severe boundary clumping in the Tutte output and downstream failure (covered separately below). The natural alternative is a target shape that cooperates with the comb's rough proportions: a rectangle matching the comb's base, capped by a half-ellipse on top. We pick the rectangle as $[0, W] \times [0, H_{\text{base}}]$ exactly — so the bottom and two short sides of the comb map to the bottom and two short sides of the dome unchanged — and pick the half-ellipse semi-axes as $(W/2, b)$ with b such that the cap arc length equals the comb's teeth-region arc length $L_{\text{teeth}} = 2.0$. This gives $b \approx 0.79$ and an arc-length-isometric mapping from the comb's teeth region



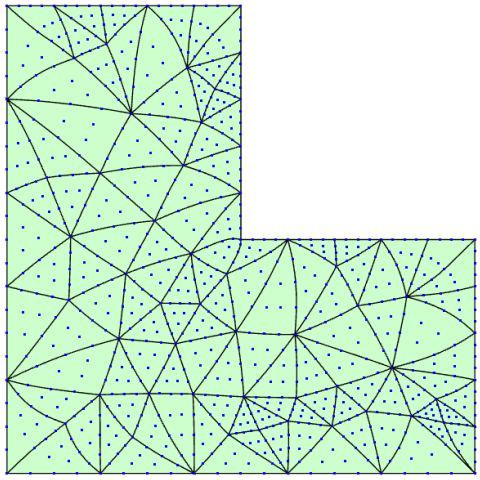
(a) Tutte (disk).



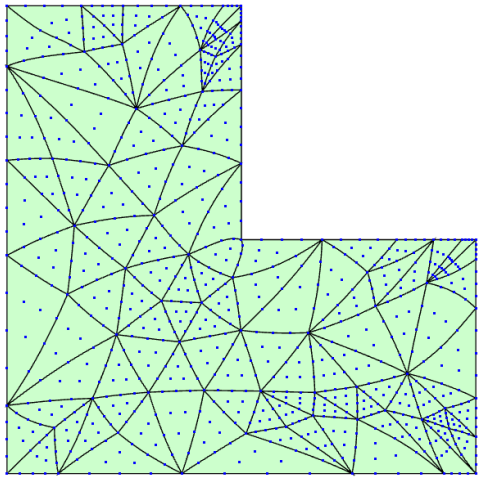
(b) (A) pinned shape-opt.



(c) (B) slid shape-opt.



(d) (A) pinned Winslow.



(e) (B) slid-disk Winslow.

Figure 14: Configurations (A) and (B) on the L-shape with the unit-disk target. Row 1: the proportional-arc Tutte disk (a), shared input to both paths. Row 2: shape optimization on the disk—(A) pins all boundary vertices at their Tutte positions (b); (B) pins only the six L-corners and lets the remaining boundary vertices slide tangentially along the unit circle (c), so shape-opt redistributes boundary spacing and creates two clumps of small reference triangles next to L-corners. Row 3: pinned Winslow on the physical L—(A) uses the original L-shape boundary as Dirichlet data (d); (B) uses proportional-arc-pulled-back L-shape positions derived from the slid disk (e). Unlike the stadium (Figure 13), the L-shape's corners give the disk-side clumping in (c) somewhere to compress into on the physical side, producing a tight cluster of near-degenerate elements at the corresponding L-corners in (e).

to the dome's cap. Each tooth gets $\sim 50°$ of cap arc instead of $\sim 20°$. The correspondence is shown in Figure 15.

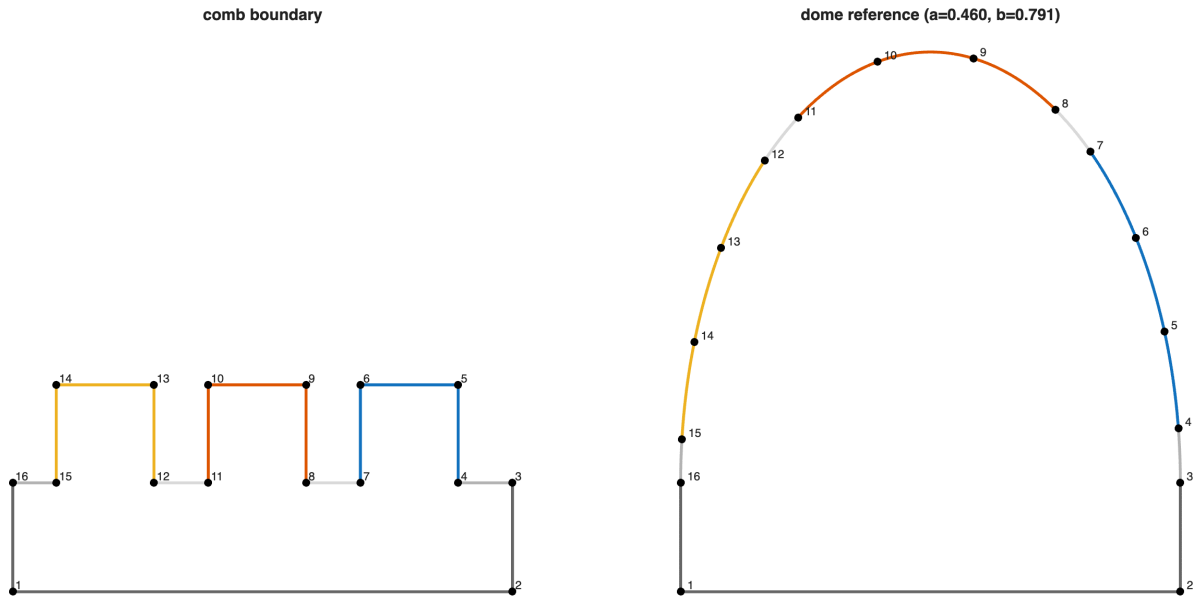


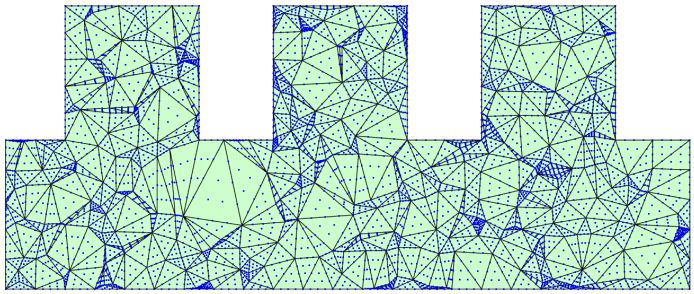
Figure 15: Boundary correspondence between the comb (left) and the dome reference (right). Same color = same edge. The bottom and two short sides of the comb (gray) map to the same straight segments of the dome rectangle. Each tooth (blue, orange, yellow) maps to a contiguous arc of the dome cap; the gaps and margins (light gray) get the small angular allotments their physical arc lengths warrant. Mapping is isometric by arc length; the arc-length-matching constraint determines the cap height $b \approx 0.79$.

The pipeline runs Tutte (with the dome positions installed as preserved boundary), shape-opt (interior only, dome boundary pinned), and direct Winslow back to the comb. We do *not* use β-homotopy continuation: the dome reference is close enough to the comb that the one-shot elliptic solve converges, and the homotopy variant locks inverted curved elements at the re-entrant tooth-base corners (linear DG-node interpolation between the dome and the comb cuts across those corners at intermediate β, producing tangled boundary nodes that the solver cannot recover from). Figure 16 shows the pipeline.

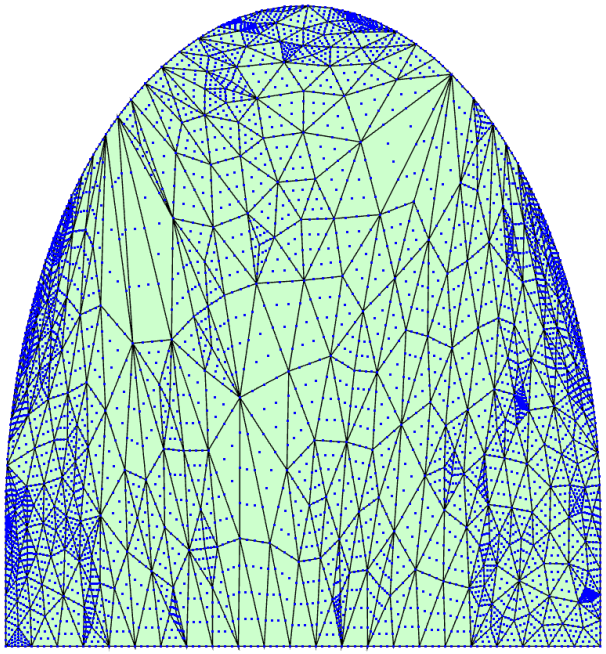
Each panel of Figure 16 fails for a specific, identifiable reason.

Why the dome Tutte clumps along the cap (b). The arc-length-isometric mapping puts about 60% of the boundary nodes on the cap (the comb's teeth-region perimeter $L_{\text{teeth}} = 2.0$ is 60% of the comb's total perimeter ≈ 3.32). The dome encloses about $2.7\times$ the comb's interior area (dome area ≈ 0.755 versus comb area ≈ 0.281), so the same 350 interior samples are spread thinner; combined with the dense cap boundary, Tutte's barycentric solve pulls interior nodes upward toward the cap. This is the same clumping mechanism we already documented for the disk target, transferred to a different shape.

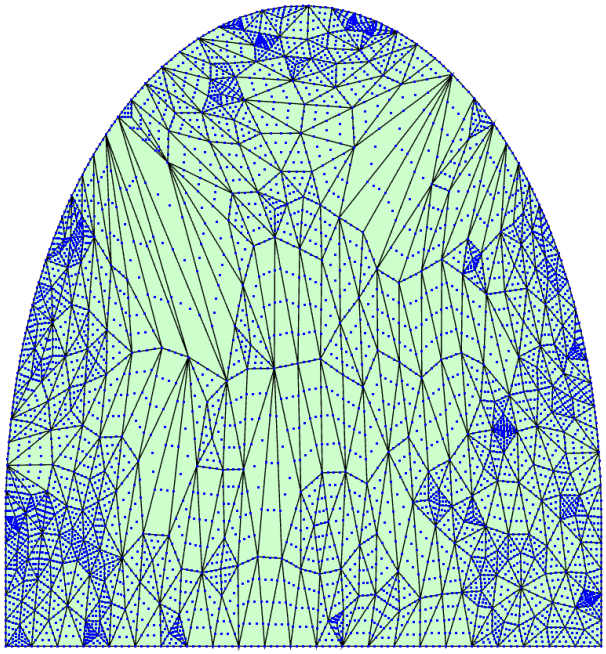
Why shape-opt does not fix it (c). The optimizer redistributes interior nodes but cannot remove boundary samples, so it converts the diffuse cap clumping into localized pockets at the four base-rectangle corners and a few residual cap spots. It also moves interior nodes toward



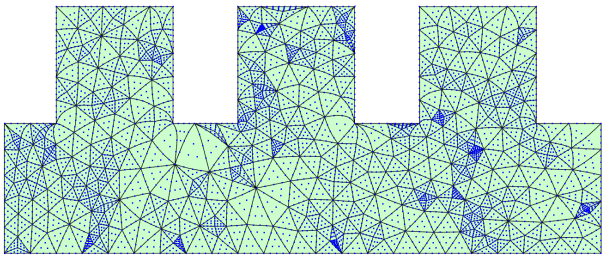
(a) Clean comb input mesh.



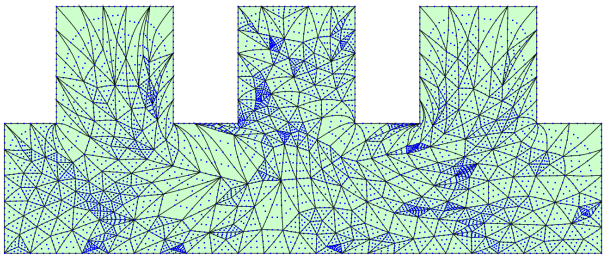
(b) Tutte to dome.



(c) Shape-optimized on dome (interior only).



(d) Direct Winslow from raw Tutte.



(e) Direct Winslow from optimized Tutte.

Figure 16: Comb pipeline with the dome reference. Top (a): clean comb input. Middle row: the raw Tutte embedding to the dome (b) and the shape-optimized version with dome boundary pinned (c). Bottom row: direct (one-shot, no β-homotopy) Winslow back to the comb, started from the raw Tutte (d) versus from the shape-optimized mesh (e). The raw-Tutte start produces the only output without inverted elements; the optimized-Tutte start produces a worse final mesh, indicating that the optimizer's local improvements on the dome reference are not aligned with the Winslow energy on the comb and degrade the physical pullback.

configurations that minimize the shape functional *on the dome* — choices that turn out to be hostile to the pullback back to the comb (e).

Why direct Winslow from raw Tutte leaves slivers at the gap walls (d). The gap walls are the comb edges between adjacent teeth — narrow corridors with re-entrant corners at top and bottom. Interior nodes that landed near the dome cap during Tutte get pulled back into these narrow corridors during the Winslow solve, and there is not room for normal-aspect-ratio triangles between the wall and the tooth body (gap_w = 0.10 versus tooth_h = 0.18), so the result is stacks of thin slivers along the walls.

Why optimized-Tutte → Winslow inverts (e). The optimizer's redistribution moved interior nodes away from the dome's corners (toward shape-opt-favorable positions on the dome). The comb's corners are exactly where the Winslow pullback most needs interior support to avoid sharp Jacobian transitions. The elliptic solve cannot reposition the displaced nodes back to where they are needed in one shot, and the result is small but real inversions at the tooth-base re-entrant corners.

Verdict. Even with a much kinder reference shape, the Tutte pipeline does not produce a fully clean mesh on the comb. Direct Winslow from raw Tutte is the only variant that avoids inversions, and even it leaves residual slivers; shape optimization on the reference actively hurts the downstream pullback. The Tutte pipeline as formulated is fragile against re-entrant corners and against the optimizer/pullback decoupling, regardless of how cooperatively the target shape is chosen.

References

[1] Meire Fortunato and Per-Olof Persson. High-order unstructured curved mesh generation using the Winslow equations. *Journal of Computational Physics*, 307:1–14, 2016.

[2] Anwei Liu and Barry Joe. Relationship between tetrahedron shape measures. *BIT Numerical Mathematics*, 34(2):268–287, 1994.

[3] A. Shi and P.-O. Persson. Local element operations for curved simplex meshes. *International Journal for Numerical Methods in Engineering*, 2023. Author manuscript.